

Marina One

Arquitetura do Sistema

SaaS Multi-Tenant com Railway + PostgreSQL

Versao 2.0 · 2026

Sumario

Secao 1	Visao Geral da Arquitetura
Secao 2	Fluxo de Requisicao HTTP
Secao 3	Multi-Tenancy — Schema per Tenant
Secao 4	Camadas do Sistema
Secao 5	Pipeline CI/CD
Secao 6	Seguranca e Autenticacao
Secao 7	Architecture Decision Records (ADRs)
Secao 8	Evolucao e Roadmap

Secao 1

Visao Geral da Arquitetura

Marina One v2.0 e uma aplicacao SaaS multi-tenant monolitica (modular). Um unico processo Node.js serve todos os tenants. O isolamento de dados e garantido por schemas PostgreSQL separados por tenant. O deploy e automatico via GitHub > Railway.

Visao de Alto Nivel



Secao 2 **Fluxo de Requisicao HTTP**

Cada request passa por **resolucao de tenant** antes de qualquer logica de negocio. O tenant determina qual schema PostgreSQL sera usado.

Sequencia completa de um request

Etapa	Acao	Detalhe
Browser	porto-belo.marinaone.com.br/api/clients	Subdominio como identificador de tenant
Cloudflare	CNAME wildcard > Railway	DNS only, sem proxy — Railway gerencia SSL
Railway	Roteamento para porta 3000	Balanceamento e health check
tenant.js	Extrai slug do subdominio	porto-belo.marinaone.com.br > slug=porto-belo
auth.js	Valida JWT Bearer token	Verifica assinatura e expiracao
compat.js	SET search_path=marina_porto-belo	Todas as queries vao para o schema correto
Route handler	Executa logica de negocio	SELECT * FROM clients (schema isolado)
Response	JSON para o browser	Dados exclusivos deste tenant

Resolucao de Tenant — Ordem de Prioridade

1. Header X-Tenant-Slug: porto-belo (NGINX/Cloudflare)
2. Subdominio: porto-belo.marinaone.com.br (mais comum)
3. Query string: ?tenant=porto-belo (fallback)
4. ENV SINGLE_TENANT_SLUG=demo (modo local)

Secao 3

Multi-Tenancy - Schema per Tenant

Padrao escolhido: Schema-per-Tenant. Cada marina tem seu proprio schema PostgreSQL (**marina_<slug>**). Isolamento total dos dados — nenhuma marina pode acessar dados de outra. O schema **saas** e global e gerencia os tenants.

Estrutura de Schemas no PostgreSQL

```
PostgreSQL (Railway)
■■■ schema: saas # Global - gerenciado pelo superadmin
■ ■■■ tenants # Lista de todas as marinas
■ ■■■ super_admins # Administradores globais
■
■■■ schema: marina_demo # Tenant local de desenvolvimento
■ ■■■ users, clients, boats
■ ■■■ spots, contracts, financial
■ ■■■ queue, shop, maintenance
■
■■■ schema: marina_porto-belo # Marina Porto Belo (producao)
■ ■■■ (mesmas tabelas)
■
■■■ schema: marina_santos # Marina Santos (producao)
■■■ (mesmas tabelas)
```

Comparacao de Estrategias de Multi-Tenancy

Estrategia	Isolamento	Complexidade	Escolhida?
Banco separado por tenant	Maximo	Alta (muitas conexoes)	Nao
Schema por tenant (atual)	Alto	Media	SIM
Coluna tenant_id	Baixo (logico)	Baixa	Nao

Provisionamento de Novo Tenant

```
# Criar novo tenant via API super-admin:
POST /api/superadmin/tenants
{
  "slug": "novo-tenant",
  "marinaName": "Marina Novo Tenant",
  "adminEmail": "admin@novotenant.com"
}

# O sistema automaticamente:
# 1. Cria registro em saas.tenants
# 2. Cria schema marina_novo-tenant
# 3. Executa todas as migrations no novo schema
# 4. Cria usuario admin inicial
```


Secao 4 Camadas do Sistema

Camada de Apresentacao (Frontend)

SPA (Single Page Application) em HTML/CSS/JS puro, sem framework. Servido pelo proprio Node.js como arquivo estatico. Comunicacao com backend via fetch() + JSON. Graficos com Chart.js (CDN).

Modulo UI	Descricao
Dashboard	KPIs, graficos de ocupacao, alertas em tempo real
Fila de Operacoes	Entrada/saida de embarcacoes, movimentacao
Clientes / Embarcacoes	Cadastro completo com historico
Contratos	Gestao de contratos de permanencia e avulsos
Financeiro	Lancamentos, contas a receber/pagar, relatorios
Loja / PDV	Venda de produtos e servicos na marina
Manutencao	Ordens de servico e historico de manutencoes
Super-Admin	Painel exclusivo para gestao de todos os tenants

Camada de Aplicacao (Backend)

Arquivo	Responsabilidade
server.js	HTTP server, registro de rotas, startup, health check
src/middleware/auth.js	JWT: geracao, validacao, hashing de senhas (SHA-256)
src/middleware/tenant.js	Resolucao do tenant por request (header/subdominio/query)
src/db/pool.js	Pool de conexoes PostgreSQL com SSL
src/db/compat.js	Helpers dbAll/dbGet/dbRun com SET search_path automatico
src/db/migrate.js	Migrations e provisionamento de schemas por tenant

Secao 5 Pipeline CI/CD

Deploy automatico: qualquer push na branch main aciona o Railway, que reconstroi a imagem Docker e substitui o container em producao. Zero downtime com health check configurado.

Fluxo de Deploy

Agente	Acao	Detalhe
Desenvolvedor	publicar.bat "descricao"	Bump versao, commit, tag, push
GitHub	Recebe o push na branch main	Webhook notifica o Railway
Railway	Clona repositorio	Detecta mudanca na branch main
Railway	docker build -f Dockerfile	Build da imagem Node.js 20 Alpine
Railway	Health check: GET /api/version	Aguarda 200 OK (timeout 30s)
Railway	Substitui container antigo	Zero downtime — novo container ativo
Railway	Deploy completo	Logs disponiveis: railway logs

Dockerfile de Producao

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

railway.json — Configuracao de Deploy

```
{
  "build": { "builder": "DOCKERFILE" },
  "deploy": {
    "startCommand": "node server.js",
    "healthcheckPath": "/api/version",
    "healthcheckTimeout": 30,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3
  }
}
```


Secao 6 Seguranca e Autenticacao

Autenticacao JWT

Aspecto	Implementacao
Algoritmo	HS256 (HMAC-SHA256) com JWT_SECRET no env
Expiracao	24 horas por token
Hash de senha	SHA-256 com salt (sem bcrypt para manter legado)
Armazenamento	localStorage no browser (token Bearer)
Renovacao	Re-login apos expiracao

Isolamento de Tenants

Mecanismo	Como Funciona
SET search_path	Cada query executa no schema do tenant — impossivel cruzar dados
JWT por tenant	Token contem o slug do tenant — validado a cada request
Super-admin separado	JWT diferente, rota /api/superadmin/* com middleware proprio
SSL obrigatorio	DB_SSL=true — conexao PostgreSQL sempre criptografada

Variaveis Sensiveis (nunca no codigo)

```
# Apenas em Railway environment variables:
DATABASE_URL # URL completa com credenciais do PostgreSQL
JWT_SECRET # Chave para assinar tokens JWT

# Nunca em:
# - codigo fonte (server.js, .js)
# - arquivos de configuracao versionados
# - CHANGELOG.md ou comentarios
```

Secao 7

Architecture Decision Records (ADRs)

ADRs documentam decisoes arquiteturais importantes, seu contexto e consequencias. Servem como referencia para futuras decisoes e onboarding de novos desenvolvedores.

ADR-01	PostgreSQL com schema-per-tenant	Aceita
Contexto	Sistema precisa suportar multiplas marinas com dados completamente isolados.	
Decisao	Usar PostgreSQL com um schema por tenant (marina_) em vez de banco separado ou coluna tenant_id.	
Consequencias	Isolamento forte, backup por schema possivel, migracao mais simples que banco separado. Custo: maior complexidade no ORM e nas queries. Limite pratico: ~100 tenants por instancia.	

ADR-02	Node.js monolito modular (sem microservicos)	Aceita
Contexto	Time pequeno, necessidade de deploy simples, sem complexidade operacional de microservicos.	
Decisao	Manter arquitetura monolitica modular em Node.js. Modularizar por feature, nao por servico.	
Consequencias	Deploy simples, debugging direto, sem latencia de rede entre servicos. Custo: escala vertical (nao horizontal por servico). Adequado para o porte atual.	

ADR-03	Railway como plataforma de deploy (PaaS)	Aceita
Contexto	Necessidade de deploy rapido sem gestao de infraestrutura (VPS, Kubernetes, etc.).	
Decisao	Usar Railway com Dockerfile + PostgreSQL gerenciado. Deploy via git push.	
Consequencias	Zero gestao de servidor, deploy automatico, PostgreSQL incluso. Custo: lock-in com Railway, limite de dominios customizados no plano Hobby (2 dominios).	

ADR-04	Frontend SPA em HTML/CSS/JS puro (sem framework)	Aceita
Contexto	Reduzir dependencias, simplificar build, facilitar manutencao por time pequeno.	
Decisao	Nao usar React/Vue/Angular. Frontend em HTML5 + CSS3 + JS vanilla, servido pelo Node.js.	
Consequencias	Zero build step, zero node_modules no frontend, carregamento rapido. Custo: sem componentizacao automatica, mais codigo repetido em UI complexa.	

Secao 8 **Evolucao e Roadmap**

Versoes Anteriores

Versao	Caracteristica Principal	Status
v1.x	Single-tenant, SQLite, deploy VPS via NGINX + PM2	Legado
v2.0	Multi-tenant SaaS, PostgreSQL, Railway, subdominios	Atual

Proximos Passos (Roadmap)

Prioridade	Item	Descricao
Alta	Notificacoes push	Alertas de vencimento de contrato e manutencao
Alta	Relatorios PDF	Exportacao de relatorios financeiros e operacionais
Media	Integracao PIX	Pagamentos online para contratos e servicos
Media	App mobile	PWA ou React Native para operadores em campo
Baixa	Redis cache	Cache de sessao e consultas frequentes
Baixa	Webhooks	Integracao com sistemas externos (ERP, contabilidade)

Quando Escalar a Infraestrutura

Indicador	Threshold	Acao Recomendada
Usuarios simultaneos	> 50	Railway Pro (US\$ 20/mes) + mais RAM
Numero de tenants	> 20 marinas	Avaliar read replicas PostgreSQL
Tempo de resposta API	> 500ms	Adicionar Redis para cache
Storage PostgreSQL	> 5 GB	Upgrade do plano Railway

Arthur Noli

Arquiteto de Software | Marina One

✉ rj.madmax@gmail.com